

# 研究型进阶专题 A · 第 1 讲

真实 Agent Benchmark 实战：第一次跑通  $\tau^3$ -bench 最小科研闭环

更新时间：2026-09-29

● **本讲结论：**选择  $\tau^3$ -bench v1.0.1 的 text half-duplex 模式，从 retail 域开始；先 1 个 task 的 Smoke Test，再扩展到 3 个、5-20 个 task。

## 1. 为什么第一次实战选 $\tau^3$ -bench，而不是更“炫”的 Benchmark

第一次做真实 Agent Benchmark，首要目标不是冲排行榜，而是建立“最低闭环能力”：你能自己安装、运行、看懂轨迹、复查评分、记录成本与版本，并从失败中提出可验证的研究观察。

| 候选                            | 优点                                                        | 第一次实战的主要负担                       | 本讲取舍 |
|-------------------------------|-----------------------------------------------------------|----------------------------------|------|
| $\tau^3$ -bench text          | 用户交互 + Tool + DB Environment + 官方 Evaluator；开源；可只跑少量 task | 存在用户模拟器，因此要学会控制一个额外随机源           | 选择   |
| BFCL V4 Agentic               | Tool calling 评估成熟，成本低                                     | 更偏工具调用能力，完整环境/用户交互弱一些            | 后续可补 |
| GAIA                          | 任务多样、知名                                                   | Web/文件/搜索工具差异较大，Harness 不如本讲目标统一 | 后续   |
| SWE-bench / Browser / OSWorld | 非常真实                                                      | 容器/浏览器/GUI/软件环境复杂，第一次容易把时间耗在基础设施 | 暂不选  |

● **科研视角：**“容易跑通”不是降低科研标准。相反，先把系统变量减少，才能更快区分 Model Failure 与 Harness / Environment / Evaluator Failure。

## 2. $\tau^3$ -bench 到底测什么？

$\tau$ -bench 系列最初就是为了评估：一个语言 Agent 能否在真实业务风格的对话里，遵守领域政策、和用户交互、正确调用工具，并把环境推进到正确的最终状态。 $\tau^2$ -bench 进一步加入双方都能操作环境的双控 dual-control 场景； $\tau^3$  版本继续扩展语音、知识检索和任务质量修复。

## $\tau^3$ -bench：一次 Episode 中的核心组件



图 1 Task、User Simulator、Agent/Harness、Tools、Environment 与 Evaluator 的关系

● **Task (任务)**：定义“要解决什么问题”。包含用户目标、初始化数据和 evaluation criteria；不是一句自然语言问题这么简单。

● **Environment (环境)**：可被工具读写的世界状态，例如订单、用户、航班、账户等数据库状态。

● **Action (动作)**：Agent 发出的 Tool Call 或给用户的文本消息。动作是系统改变世界的接口。

● **Harness (运行框架)**：把 Task、Agent、User、Tool、Environment 串起来的控制逻辑。 $\tau^3$ -bench 中 orchestrator 负责轮转、执行工具、终止和记录。

● **Evaluator (评测器)**：把完整 trajectory 映射为 reward。对 retail/airline/telecom，官方默认主要由 DB 与 COMMUNICATE 两类条件门控。

● **容易误解**：`evaluation\_criteria.actions` 不是“Agent 必须逐项照着做”的标准答案。官方文档明确：在 retail/airline/telecom 中，它通常只是生成目标 DB 状态的一条参考轨迹；Agent 走另一条正确路径，只要最终 DB 状态等价且必要信息传达正确，也可以得满分。

### 3. 一个 Episode 是怎样运行的？

Episode 运行流程：从 Task 到 Reward



图 2 一个 text half-duplex Episode 的最小运行闭环

Episode 可以理解为“一个 task 的一次完整尝试”。同一个 task 重跑 3 次，就是 3 个 episode / trials。真实 Agent 评测不能只看一次，因为模型、用户模拟器和 API 都可能产生随机性。

1. Harness 读取一个 Task，并创建该 task 的初始 Environment。
2. User Simulator 按任务说明生成用户消息。
3. Agent 读取 policy、对话历史和 tool schema，选择回复或调用工具。
4. Harness 执行 Tool Call，Environment 返回 Tool Result，并可能改变数据库状态。
5. User 与 Agent 继续交互，直到正常停止、超时、达到最大步数或出现错误。
6. Evaluator 用 trajectory 与 task criteria 计算 reward，并把 cost、duration、termination、messages 等记录进 results.json。

### 4. Evaluator：成功到底是怎么判的？

| RewardType    | 检查对象                | 零基础理解                                           |
|---------------|---------------------|-------------------------------------------------|
| DB            | 最终数据库状态             | 事情最终办对了吗？                                       |
| COMMUNICATE   | 要求说给用户的信息           | 该告知的内容说了吗？                                      |
| ACTION        | 具体工具调用是否匹配参考 action | 是否走了指定动作路径；标准 retail/airline/telecom 通常不把它作为硬门槛 |
| ENV_ASSERTION | 环境断言                | 最终状态是否满足显式条件                                    |
| NL_ASSERTION  | LLM 判断自然语言断言        | 实验性/WIP；不要当成绝对真值                                |

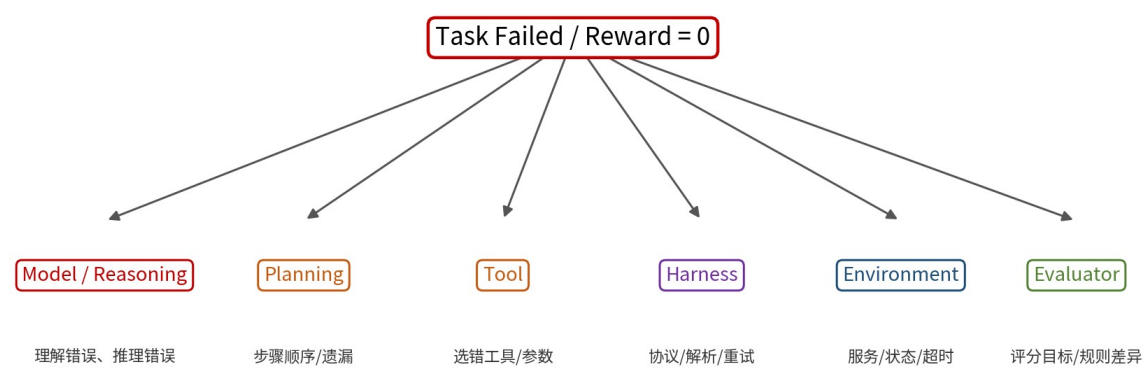
对于常见的 `reward\_basis = [DB, COMMUNICATE]`，可以把最终 reward 理解为两个门都要过：最终状态正确，并且必要信息传达正确。只要其中一个为 0，整体就可能为 0。

🔴 **科研洞察：** Evaluator 本身也是系统组件。Benchmark 分数不是“模型本体能力”的纯净测量，它是 Model × Prompt × Harness × User Simulator × Environment × Evaluator × API Runtime 的联合结果。

## 5. Trace：为什么它比总分更重要？

总分只能告诉你“最后失败了”，Trace 才能回答“从哪里开始变坏”。本讲把 Trace 当作科研观测数据，而不是调试垃圾。

Failure 归因树：不要把所有失败都叫“模型不行”



First Bad Span = 最早出现、并能解释后续失败传播的关键异常片段

图 3 Failure 归因树与 First Bad Span

| Failure 类别        | 典型证据                  | 不要误判为           |
|-------------------|-----------------------|-----------------|
| Model / Reasoning | 读懂了工具返回，但做出错误结论       | Harness failure |
| Planning          | 漏步骤、顺序错误、提前结束         | 工具本身坏了          |
| Tool              | 工具选错、参数非法、调用结果错误      | “模型智力低”         |
| Harness           | 消息协议、解析、重试、状态传递异常     | 模型失败            |
| Environment       | 服务超时、初始状态异常、数据库/模拟器问题 | 规划失败            |
| Evaluator         | 实际完成但评分条件/任务定义不一致     | 模型失败            |

● **First Bad Span**: 不是最后一条失败消息，而是“最早出现、且能解释之后失败传播”的关键片段。比如第 3 步把 order\_id 看错，后面的错误退款只是后果。

## 6. Run Manifest：为什么论文级实验必须记录版本

如果三周后你无法回答“当时到底用了哪个 benchmark commit、哪个模型别名、哪个 seed、哪条命令”，那次实验就不能称为可复现科研证据。

| 必须记录                                    | 为什么                            |
|-----------------------------------------|--------------------------------|
| Benchmark tag + git commit              | task / evaluator / harness 会变化 |
| Agent model + provider + reasoning args | 模型别名可能更新，参数影响策略                |

|                                |                               |
|--------------------------------|-------------------------------|
| User simulator model           | 它也是随机环境的一部分                   |
| Task IDs + split               | 避免 task 集合漂移                  |
| Seed + trials                  | 支撑重复实验                        |
| Prompt / Harness version       | 避免“模型不同”其实是 prompt/harness 不同 |
| Token / Cost / Latency / Retry | 控制 compute 与基础设施差异            |
| Raw Trace + results SHA256     | 保留原始证据链                       |

## 7. 当前低成本模型与预算怎么估

本讲示例默认使用 GPT-5.6 Luna 只因为它在 2026-09-29 的 OpenAI API 价格较低，且支持 function calling；Benchmark 本身不绑定 OpenAI， $\tau^3$ -bench 通过 LiteLLM 可接其他 provider。

| 模型                  | 当前公开文本价格（每 1M tokens）         | 本讲定位                |
|---------------------|-------------------------------|---------------------|
| GPT-5.6 Luna        | \$0.20 input / \$1.20 output  | Smoke / Pilot 低成本起步 |
| GPT-5.6 Terra       | \$2.00 input / \$12.00 output | 需要更强模型时再比较          |
| 其他 LiteLLM provider | 以各 provider 当日价格为准            | 后续跨模型验证             |

举例：假设一个 episode 合计（Agent + User Simulator）消耗 100k input + 10k output tokens，Luna 的粗略 token 成本约为  $0.02 + 0.012 = 0.032$  美元。20 个同量级 episode 约 0.64 美元。这个只是预算例子，不是  $\tau^3$ -bench 的“固定每 task 成本”；真实成本以 results.json 中的 cost 与 token usage 为准。

● **实验坑：**不要用“20 个 task 大约几美元”代替真实记录。历史对话会重复进入 prompt，复杂 task 的 token 可能远高于简单 task；Retry、User Simulator、LLM review 也会额外花钱。

## 8. 第一次真实运行：严格按 1 → 3 → 5-20 的顺序

### Step 1：固定官方版本

```
git clone --branch v1.0.1 --depth 1 https://github.com/sierra-research/tau2-bench.git
cd tau2-bench
uv sync
uv run tau2 check-data
git rev-parse HEAD
git describe --tags --always
```

Ubuntu 22.04 自带 Python 3.10 不必手工折腾系统 Python。官方 v1.0.1 使用 uv，`.python-version` 固定 3.12，uv 可以自动下载兼容 Python。

### Step 2：只配置 API Key，不写进代码

```
cp .env.example .env
# 用 VS Code 打开 .env，使用 provider 的 Key。
# 绝对不要把 .env 提交到 GitHub，也不要论文附件中公开。
```

### Step 3 : 1 ↑ task Smoke Test

```
uv run tau2 run \
  --domain retail \
  --agent-llm openai/gpt-5.6-luna \
  --user-llm openai/gpt-5.6-luna \
  --num-trials 1 \
  --num-tasks 1 \
  --max-concurrency 1 \
  --seed 300 \
  --timeout 300 \
  --verbose-logs \
  --llm-log-mode all \
  --save-to a01_smoke_retail
```

● **Smoke Test 的成功标准**：不是 reward 必须等于 1，而是：程序完整跑完、生成 results.json、能用 tau2 view 打开、能看到消息和 Tool Call、能确定 termination\_reason、分析脚本能读结果。

### Step 4 : 3 ↑ task，再到 5 个 task

只有 Smoke Test 的基础设施稳定后，才扩大任务量。第一次不要用高并发，因为并发会引入 rate limit、排队和 retry，干扰你理解单个 episode。

### Step 5 : 看官方 Trace

```
uv run tau2 view --file data/simulations/a01_smoke_retail/results.json
```

不要只盯 reward。顺着消息看：用户说了什么 → Agent 为什么调这个工具 → 参数是什么 → 工具返回什么 → Agent 下一步如何更新判断。

### Step 6 : 运行本资料包的分析脚本

```
python /path/to/资料包/src/analyze_tau3_results.py \
  data/simulations/a01_smoke_retail/results.json \
  --out-dir /path/to/资料包/results/real_smoke
```

## 9. 从 Failure 到 Research Observation

从 Benchmark Failure 到 Research Question



图 4 不从“先想新方法”出发，而从可重复的 Trace 现象出发

7. 先标 First Bad Span，不急着给原因。
8. 把同类失败聚类：错误工具选择？无效参数？过度沟通？错误确认？提前终止？
9. 至少在多个 task / trial 中确认它不是单例。

10. 提出可证伪的解释，例如“长对话导致旧约束被忽略”。
11. 设计控制变量：保持模型、task、token budget 不变，只改变被怀疑的因素。
12. 如果效应稳定，再形成 Research Question；如果不稳定，就否决。

● **一个真正可研究的例子：**如果你观察到：同一模型在需要 6-8 次 tool interaction 的任务中，经常在一次正确读取后又重复查询并覆盖先前状态；而短任务很少发生。下一步研究不是立刻发明“新 memory 模块”，而是先验证“任务交互长度 → 状态漂移/冗余动作”的现象是否稳定。

## 10. 20 必能回答的

- Benchmark 测什么？
- Task 长什么样？
- Environment 是什么？
- Agent 能做什么 Action？
- Tool 是什么？
- Harness 是什么？
- Evaluator 怎么判成功？
- Metric 如何计算？
- Episode 如何运行？
- Trace 在哪里？
- 怎么保存 Trace？
- 怎么找 First Bad Span？
- 如何区分 Model/Planning/Tool/Harness/Environment/Evaluator Failure？
- API Cost 怎么记录？
- Token 怎么记录？
- Latency 怎么记录？
- Retry 怎么记录？
- Model/API version 怎么记录？
- Prompt/Harness version 怎么记录？
- 如何从 Failure 走到 Research Question？

## 11. 本讲完成标准

| 层级   | 你做到什么才算过关                                                      |
|------|----------------------------------------------------------------|
| 最低闭环 | 1 个真实 task 跑完 + results.json + tau2 view + Run Manifest        |
| 合格   | 3-5 个 task + cost/token/duration 汇总 + 至少 1 个失败的 First Bad Span |
| 研究起点 | 5-20 个 task + 失败分类 + 发现一个可重复 Observation + 写出一个 falsifier      |

## 12. 官方资料与一手来源

$\tau^3$ -bench /  $\tau$ -bench GitHub: <https://github.com/sierra-research/tau2-bench>

Release v1.0.1: <https://github.com/sierra-research/tau2-bench/releases/tag/v1.0.1>

Getting Started v1.0.1: <https://github.com/sierra-research/tau2-bench/blob/v1.0.1/docs/getting-started.md>

Evaluation v1.0.1: <https://github.com/sierra-research/tau2-bench/blob/v1.0.1/docs/evaluation.md>

CLI Reference v1.0.1: <https://github.com/sierra-research/tau2-bench/blob/v1.0.1/docs/cli-reference.md>

$\tau$ -bench paper: <https://arxiv.org/abs/2406.12045>

$\tau^2$ -bench paper: <https://arxiv.org/abs/2506.07982>

OpenAI GPT-5.6 Luna model page: <https://developers.openai.com/api/docs/models/gpt-5.6-luna>

● **版本说明:** 资料按 2026-09-29 检索。若未来官方 release 更新, 请先读 CHANGELOG, 再决定是否升级; 论文主实验不要中途偷偷升级 task/evaluator。